

RETO HEALTH — Guía paso a paso para construir en Claude Code

Qué construimos aquí: **web pública + marketplace + lobby de cliente (login) + admin**, en 3 regiones (US/LATAM/EU), con Stripe para el libro y QuickBooks para cámara/protocolo. **Sin datos clínicos / sin PHI** — eso es una fase posterior con compliance aparte.

Documento de arquitectura de referencia: `PLAN_reto_portal.md`. Esta guía es la de ejecución.

Reglas de oro (léelas una vez)

1. **Carpeta del proyecto en** `~/code/reto-portal`, NO en Desktop ni en nada sincronizado con iCloud (te ralentiza Claude Code, ya te pasó).
 2. **Dónde escribes cada cosa:**
 - Prompt de Claude Code (empieza con `>`) → ahí pegas los prompts de esta guía y le hablas al agente.
 - Terminal / zsh (acaba en `%`) → ahí van comandos de shell (`cd`, `pnpm`, `git`, `open`).
 - No pegues comandos de shell en el prompt de Claude.
 3. **Secretos** → solo en `.env.local`. Nunca a git, nunca me los pegues a mí. Si subes uno por error, rota la llave.
 4. **Bloque a bloque:** no pases al siguiente hasta validar el CHECKPOINT.
-

PARTE 0 — Cuentas y llaves que necesitas antes de empezar

Crea estas cuentas (las que no tengas) y ten a mano dónde sacar las llaves. Claude Code te las irá pidiendo bloque a bloque; no hace falta tenerlas todas el día 1, pero crea las cuentas ya.

Servicio	Para qué	Qué necesitarás
GitHub	repo / deploy	cuenta + repo vacío <code>reto-portal</code>
Supabase	DB + Auth (comercio)	Project URL, anon key, service_role key, DATABASE_URL, DIRECT_URL
Stripe	pago del libro	test: publishable + secret keys, webhook secret
Intuit Developer + QuickBooks Online	facturas/links cámara+protocolo, recibos del libro	app en developer.intuit.com (sandbox), client id/secret, company sandbox
Resend	emails (links de pago, confirmaciones)	API key + dominio remitente
Vercel	hosting	cuenta conectada a GitHub

Activos: el **logo PNG** y **fotos de la cámara + portada del libro**. Y el archivo de diseño `RETO_Health_Storefront.html`.

Pendiente que decides: **precio de la cámara en LATAM** (si no lo tienes, lo dejamos como "bajo consulta").

PARTE 1 — Arranque del proyecto (en la terminal)

Paso 1 — Comprueba versiones (en terminal):

```
node -v      # 20+ ideal
pnpm -v     # si no está: npm i -g pnpm
```

Paso 2 — Crea la carpeta y entra:

```
mkdir -p ~/code/reto-portal && cd ~/code/reto-portal
```

Paso 3 — Mete los activos dentro (crea una carpeta y copia ahí el logo, las fotos y el HTML de diseño):

```
mkdir -p _design
```

Copia a `~/code/reto-portal/_design/` el `RET0_Health_Storefront.html`, el logo y las fotos.

Paso 4 — Abre Claude Code en esta carpeta:

```
claude
```

Paso 5 — Ya dentro de Claude Code (prompt `>`), pega el **Bloque A** de abajo.

PARTE 2 — Bloques de build

En cada bloque: pegas el PROMPT en Claude Code, dejas que trabaje, haces lo que diga "Tú haces", y no sigues hasta el CHECKPOINT.

Bloque A — Fundación + identidad

PROMPT:

Vamos a construir un portal para RET0 Health. Stack: Next.js 15 (App Router) + TypeScript + Tailwind + Supabase + Stripe + QuickBooks Online + Resend. Hosting Vercel.

Bloque A – fundación, NO toques pagos ni auth todavía:

1. Inicializa Next.js 15 con TypeScript, App Router, Tailwind, pnpm.
 2. En `_design/` tienes `RET0_Health_Storefront.html` (diseño aprobado) + logo + fotos. Extrae los design tokens y créalos como CSS variables globales (negro `#000000`, superficies `#08080A`–`#151518`, ink `#F4F4F2`/`#B6B6B4`/`#87878A`, accent `#EDEF0`, gradiente iridiscente del logo, `--positive #8FE0B0`, `--warning #E8C98F`; fuentes Helvetica Neue + IBM Plex Mono). Mete el logo en `/public`.
 3. Routing por región: `/us`, `/latam`, `/eu`. i18n con diccionario `en/es` (US=`en`, LATAM/EU=`es`). Config de región única que define moneda (USD/USD/EUR), idioma y precios.
 4. Layout base (header con logo + selector de región, footer) fiel al diseño, fondo negro.
- Para. No avances al resto hasta que yo valide.

Tú haces: nada aún (no pide llaves todavía). **CHECKPOINT:** corre `pnpm dev`, abre `/us` `/latam` `/eu` → home en negro con logo y el selector cambia idioma/moneda.

Bloque B — Web pública (portar el diseño)

PROMPT:

Bloque B: implementa la web pública portando el diseño de `_design/RETO_Health_Storefront.html` a componentes de Next, usando los tokens del Bloque A. Secciones: hero, propuesta de valor, ciencia/tecnología, prueba social (atletas), teaser de productos, CTA. Copy de marca: `longevidad/biohacking/recuperación/rendimiento, Miami+Madrid, "Strength · Recovery · Clarity"`. Responsive móvil primero. Sin backend.

CHECKPOINT: la home se ve fiel al diseño en móvil y desktop, en las 3 regiones.

Bloque C — Marketplace (catálogo)

PROMPT:

Bloque C: marketplace.
Productos:
– LIBRO "El Método RETO": US \$29.99 / EU €29,99 / LATAM \$29.99 USD. Compra directa on-site.
– CÁMARA HIPERBÁRICA: US \$64.900 / EU €69.900 / LATAM "Precio bajo consulta". NO es compra directa: botón "Solicitar" que abre un formulario de solicitud (nombre, apellidos, email, teléfono, país, dirección de envío) y guarda un LEAD.
Crea: página tienda (grid; cámara como producto estrella, libro secundario), ficha de producto de cada uno, y el formulario de solicitud de la cámara. Aún sin pago ni persistencia real (mock).

CHECKPOINT: tienda y fichas correctas; precios por región bien; el botón de la cámara abre el formulario de solicitud.

Bloque D — Checkout del libro con Stripe (test)

PROMPT:

Bloque D: checkout SÓLO del libro, en Stripe TEST mode.

- Formulario de datos antes de pagar: nombre, apellidos, teléfono, email, fecha de nacimiento (DOB), dirección de facturación y de envío (con toggle "igual que facturación").

- Pago con Stripe Payment Element (la tarjeta va directa a Stripe; NUNCA guardamos el número). Crea PaymentIntent en server. Guarda solo Stripe Customer id + PaymentMethod token.

- Página de confirmación.

Dime exactamente qué llaves de Stripe necesitas y de dónde las saco.

Tú haces: cuando lo pida, saca de Stripe (modo Test) la publishable key, la secret key y, para el webhook, el signing secret. Mételas en `.env.local` (o donde Claude te indique). No me las pegues a mí. **CHECKPOINT:** compra de prueba con tarjeta `4242 4242 4242 4242` → confirmación OK, y en `.env.local` no hay ningún número de tarjeta.

Bloque E — Persistencia (Supabase, comercio)

PROMPT:

Bloque E: persistencia en Supabase (proyecto de COMERCIO, no clínico).

Diseña el schema sync-friendly (IDs limpios, timestamps): customers (+ stripe_customer_id), addresses, products, prices(product, region, currency, amount nullable), orders, order_items, payments(stripe_pi_id, stripe_pm_id, status), leads(cámara: datos + estado), qb_docs(order/lead, tipo, qbo_id, status).

Guarda el pedido del libro y los leads de la cámara. RLS básico. Dime qué llaves de Supabase necesitas y de dónde.

Tú haces: crea el proyecto en Supabase, saca Project URL + anon key + service_role key + DATABASE_URL (Transaction, puerto 6543) + DIRECT_URL (Session, 5432). Si no encuentras el password de DB, en Settings → Database puedes resetearlo. Mételas en `.env.local`.

CHECKPOINT: una compra de libro y una solicitud de cámara aparecen como filas en Supabase.

Bloque F — QuickBooks (recibos + links de pago)

PROMPT:

Bloque F: integración QuickBooks Online (OAuth2, SANDBOX primero).

- LIBRO: cuando el pago de Stripe se confirma (webhook), crea un SalesReceipt en QBO (pago ya cobrado) y guarda el qbo_id.
 - CÁMARA y PROTOCOLO: función para crear una Invoice en QBO con pago online habilitado y enviarla por email al cliente (link de pago). Para la cámara se dispara desde el lead en admin; el protocolo se creará desde admin/pipeline.
- Nota: enviar links de pago reales requiere QuickBooks Payments conectado en la cuenta de producción; en sandbox simúlalo. Dime qué credenciales de Intuit necesitas y de dónde.

Tú haces: crea una app en developer.intuit.com, modo sandbox; saca client id + client secret + redirect URI; conecta la company sandbox. Mételas en `.env.local`. **CHECKPOINT:** una venta de libro genera un SalesReceipt en el QBO sandbox; puedes generar una Invoice de cámara con link de pago (sandbox).

Bloque G — Auth + lobby de cliente (login only)

PROMPT:

Bloque G: área de cliente ("lobby") con Supabase Auth.

- SOLO login. NO existe registro público ni botón "crear cuenta".
- Tras login, un lobby mínimo detrás de auth (placeholder; aquí irán resultados y módulos clínicos en el futuro). Sin suscripciones, sin historial de compras, sin productos.
- Middleware que protege /app y redirige a login si no hay sesión.

CHECKPOINT: sin cuenta no entras; con un usuario de prueba (créalo a mano en Supabase) entras al lobby; no hay forma de auto-registrarse.

Bloque H — Provisión de cuentas

PROMPT:

Bloque H: creación de cuentas de cliente (dos vías, nunca auto-registro):

1. Manual: desde admin, crear cuenta de cliente (email) y enviar invitación/acceso por email (Resend).
2. Automática: cuando se paga el PROTOCOLO (Invoice de QB0 marcada como pagada → webhook/sync), crear automáticamente la cuenta del cliente y darle acceso al lobby.

CHECKPOINT: marcar una invoice de protocolo como pagada en sandbox crea la cuenta y el cliente puede entrar; y puedes crear una cuenta manualmente desde admin.

Bloque I — Admin

PROMPT:

Bloque I: área de admin en subdominio admin. (mismo codebase, route-group + RBAC; rol admin en el JWT).

Funciones: ver/gestionar pedidos del libro, ver leads de cámara y generar su Invoice/link de QB, gestionar productos y precios por región, crear cuentas de cliente (Bloque H), ver estado de QuickBooks. Middleware estricto para admin.

CHECKPOINT: desde admin gestionas pedidos, leads, productos y creas cuentas; un usuario sin rol admin no entra.

Bloque J — Pulido + deploy

PROMPT:

Bloque J: responsive final fiel al diseño, estados de error/carga, y prepara deploy a Vercel. Lista las variables de entorno que hay que configurar en Vercel.

Tú haces: conecta el repo a Vercel, mete las env vars, deploy a la URL de Vercel. Prueba todo en esa URL. **CHECKPOINT:** el portal funciona end-to-end en la URL de Vercel (test mode).

PARTE 3 — Salir a producción (cuando todo esté validado)

1. Stripe: pasa de test a **live keys** (rota el webhook).

2. QuickBooks: pasa de sandbox a la **company real** y conecta **QuickBooks Payments** (necesario para los links de pago reales; recuerda que QB Payments es de EE.UU. — para EU/LATAM confirmamos si hace falta cobrar por Stripe).
 3. Dominio: NO toques el apex (`retohealth.com`) todavía si la web actual de Square sigue viva — lanza primero en un subdominio (p.ej. `app.retohealth.com`) y migras el apex cuando estéis listos, para no tumbar el sitio actual.
-

Pendientes que cierras tú

- Precio de la cámara en LATAM.
- ¿`retohealth.com` directo o subdominio para el portal?
- Confirmar hex de acento y fuente sans definitivos de marca (si los hay) — ahora va monocromo.
- Para EU/LATAM: ¿QuickBooks Payments o Stripe para el cobro real?

Fase clínica (resultados + HIPAA) = proyecto aparte. No la empieces sin BAA con los proveedores y la base clínica aislada. Te preparo el roadmap HIPAA en cristiano cuando lleguemos.